TODOs

- [x] ~~Refine # of agents / remove "agent"s from the slides~~
- [x] ~~Plan/create full customer profile (to convey how different users use it during demo)~~
- [ ] Challenge technical questions
- [x] ~~CODEEEEEE~~
- [ ] LEARN SCRIPTS
- [x] ~~Implementation flow slide: why do we need to go through?~~

CODEEEEE

- [x] ~~Actions flow when we REJECT step~~
    - reject -> pop up a box asks why/do you want to create a new plan -> get to the reasoning/planing layer again (info about the previous risk case is saved so Manager Agent can just go back to previous layer) -> …
- [x] ~~In Risk Simulation page, we want "The iteration history stays visible for comparison across different runs." Currently, it is not (just showed that the plan has been rejected without showing the details.~~
- [x] ~~Risk Simulation and chatbot are generating very slowly. Try?~~
- [x] ~~Validate supply chain knowledge of the reasoning layer~~
- [x] ~~Extend the Actions tab so the agent doesn't stop at preparing documents but also plans delivery routes via interactive maps?~~
- [x] ~~Refactor our agents (to the new architecture)~~

TECHNICAL QUESTIONS

Q	A
Why this current stack? Pros & cons? Efficiency?	- frontend: **React** gives us component reusability across the approval work flow, risk dashboards, and simulation views. **Vite** because build times are faster. **Cons**: Vite's ecosystem is younger and some enterprise UI libraries assume webpack - backend: **Python** is the natural home for AI/ML orchestration (google adk, data processing libraries, etc.) **FastAPI** is critical when we're making multiple concurrent LLM API calls per request. **Cons**: Python's GIL limits true parallelism for CPU-bound asks; but our bottleneck is I/O bound LLM calls so this doesn't hurt in practice - database: **Supabase** gives us **PostgreSQL** with built-in row-level security, real-time subscriptions, and a REST API layer out of the box. Row-level security is directly relevant to our multi-tenant isolation requirement → can enforce tenant boundaries at the database layer, not just the application layer. **Cons**: Superbase is a managed service and we trade infrastructure control for convenience. Customer is now on shared Supabase infrastructure. For

	customers requiring dedicated infrastructure, that's a VPC deployment conversation. - agent: 1) We're building on **Google ADK** which has native **Gemini** integration. 2) Gemini's context window is large enough to hold a full supplier network profile plus event context i na single call, which matters for our Hypothesis Agent. **Cons**: vendor concentration risk: our orchestration layer, LLM, and hosting are all Google. If we were building for production resilience, we'd abstract the LLM layer to support fallback to Claude or GPT. Note: - "VPC" is virtual private cloud - "vendor concentration risk" means "the danger of operational disruption and financial loss caused by excessive reliance on a small number of suppliers for critical goods or services"
What are the considerations when scaling up the project?	Database: - Supabase's tiers have connection limits → at production scale we add PgBouncer connection pooling - We partition the database by tenant ID from day one → horizontal sharding is possible without schema changes when we need it Backend: - Fast API scales horizontally behind a load balancer. - Harder problem is LLM API rate limits. Gemini's rate limits are per-project, not per-instance so adding more backend instances doesn't increase throughput unless we also implement a request queue - We'll add a job queue like Google Cloud Tasks between the FastAPI layer and the ADK orchestration layer, so LLM calls are rate-limited gracefully rather than failing under load. Agent orchestration: Frontend: - Scaling concern is real-time updates: if the customer has many active risk cases simultaneously, we need WebSocket connections for live status updates. Supabase's real-time subscriptions handle this natively, but at high concurrency, we'd move to a dedicated pub/sub layer
Technical risk mitigations? READ THE SLIDE (attached below) and note relevant technologies	
How to ensure the accuracy or trustworthy of the	Structured output enforcement: - Every agent outputs against a defined Pydantic schema. If the output doesn't conform, the call fails and retries. We never pass malformed agent output

agents (in terms of generating plans)	downstream. This eliminates a class of hallucination where the model invents fields Confidence propagation: - Uncertainty is explicit at every stage. The Normalizer Agent outputs a geo_confidence field. The Hypothesis Agent outputs a confidence score per causal link. The Scoring Agent propagates these into the final risk score. Customer will see a confidence-adjusted score, not a false precision number Evaluation harness: - (future implementation) Maintain a library of historical supply chain incidents with known outcomes. Every model update or prompt change runs against this library before deployment. Regression threshold: if causal direction accuracy drops below 70% on the test set, the deployment is blocked Human verification surface: - Every plan shows its full reasoning chain: which data points were used, which causal links were inferred, which memory patterns were matched. - Customer can trace recommendation back to its source.
Database capacity	Current schema has 5 primary tables: events, risk cases, plans, memory records, and audit log. At 100 customers with 300 events per day each → 30000 event records per day → 10M per year. This is manageable for PostgreSQL The vector store is the capacity concern: each supplier entity and each pattern memory record generates an embedding, roughly 1536 dimensions → 6KB per record. At 10K memory records per customer across 100 customers = 6GB of vector data, which is still within a comfortable range. However, as we scale larger with more customers and deeper memory…
Infrastructure for running?	
Normalizer Agent outputs a Canonical Schema. Show that schema. What fields does it have, what happens when a GDELT event has no geographic coordinates, and how do you handle conflicting severity ratings from two different feeds on the same event?	

Hypothesis Agent generates causal chains. What model are we using, what's our prompt structure, and how do we prevent it from generating a confident causal chain on insufficient evidence?	
Why is the Scoring Agent a separate agent and not just a deterministic function? Risk Score = Probability × Exposure × Impact is a formula. You don't need an LLM for that. Defend the decision.	- The formula itself is deterministic. But the inputs to the formula benefit from reasoning - Where the LLM earns its place is in estimating Probability, which requires interpreting the causal chain Hypothesis Agent produced and contextualizing it against Pattern memory - Impact estimation also requires judgement since the same port closure has different impact on a company with 3 days buffer inventory vs 30
Your Memory Layer has three stores. What's the underlying technology for each? Vector DB, relational, graph? And why different stores instead of one unified store?	
What's your ground truth? How do you know when your Hypothesis Agent is wrong? If the client approves a bad recommendation and loses money, how does your system know it made a mistake and how does that feed back?	
Exposure Agent maps events to suppliers, routes, and inventory. How to control how fresh the mapping data is? What happens	

when a supplier changes their warehouse location and nobody tells Omni (will this happen?)?	
You require human approval before ERP commits. In a fast-moving crisis: port closure, imminent hurricane, etc. what's your expected time from signal detection to client seeing a ranked plan? Give me actual numbers.	
If the client approves 95% of what the system recommends, has she stopped thinking critically or has the system earned her trust? How do you design against automation bias?	Automation bias mitigation: - We never show a single recommendation. Always show a ranked list of at least three options with tradeoffs made explicit, so client is forced to make a choice rather than approve a suggestion - The causal chain is always visible. The client can't approve a plan without seeing the reasoning behind it - We track the client's override rate in their dashboard and surface it to them (ask for feedback/notification)
# of agents. Can we optimize?	
How is Pattern Memory stored and retrieved?	Storage: - memory_patterns table (pattern id, triggers, recommended/avoid actions, cost/benefit stats, confidence) Retrieval: - Included in live context (agent_runner) for every run - Queried by manager tools (check_memory_patterns, query_patterns) when deciding what to do. Update: - Reflection writes back lessons as new/updated patterns via update_from_reflection, slowly increasing confidence and enriching recommended/avoid actions over time.
Current cloud solution?	At this stage, infrastructure is not our differentiator but our agent design and domain logic are. Managed services let us move fast and prove the intelligence layer works before investing in infrastructure we don't need yet.

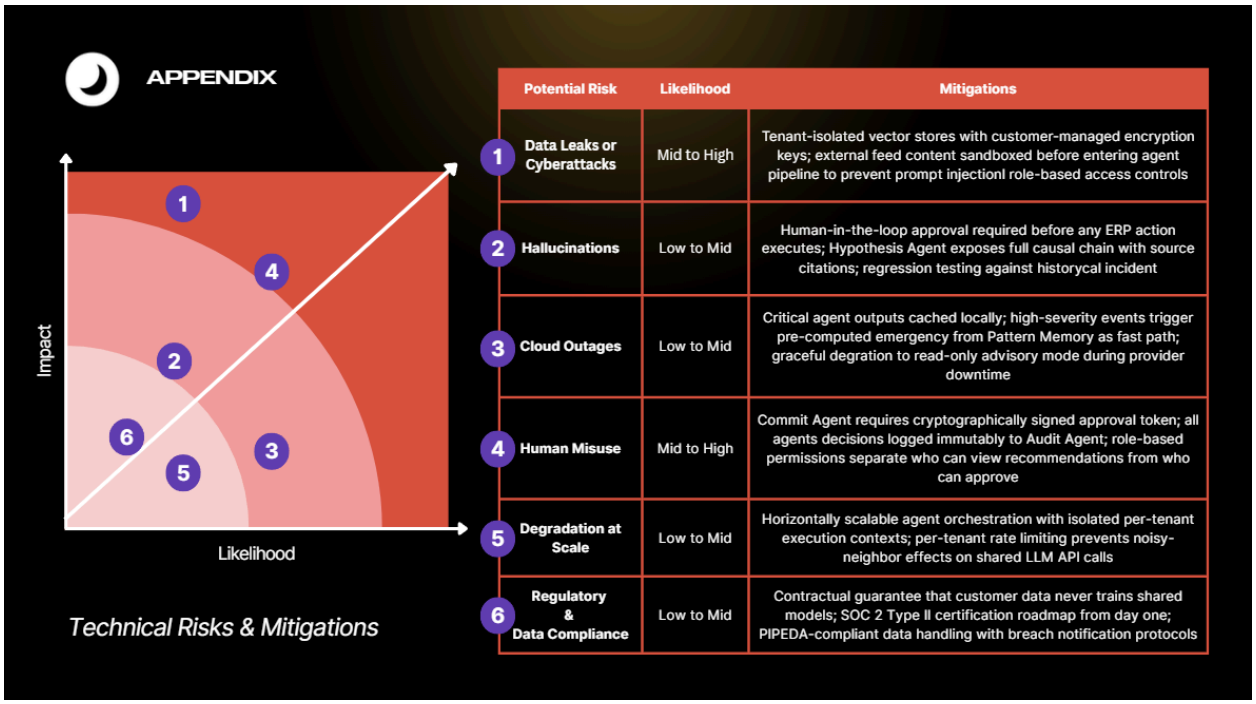
What's your cloud migration path as you scale?	3 stages that map to company growth 1. Current: - Pure managed services: Supabase, Gemini API, third-party feeds. Zero infrastructure overhead, fast iteration 2. Containerization when gaining customers (10+): - FastAPI backend and React frontend move into Docker containers deployed on Cloud Run or equivalent. - Supabase stays managed but we add connection pooling and read replicas. - Introduce a job queue on Cloud Tasks between API layer and agent orchestration so LLM calls are rate-limited gracefully rather than failing under concurrent load 3. VPC deployment - Customer's own cloud account, our containers deploy there, their data never leaves their environment - Supabase self-hosted or RDS as the database layer
You're entirely dependent on third-party APIs. What happens if Supabase goes down, Gemini has an outage, or GDELT changes their API?	Supabase outage: - Our read-heavy operations like pattern retrieval or entity lookups can be served from an in-memory cache for short outages. - Write operations queue and retry. - Critical risk cases already displayed to customer remain visible because the frontend holds state Gemini outage: - pre-computed playbooks from Pattern Memory are served as immediate directional guidance without LLM involvements. - The system degrades gracefully to rule-based recommendations rather than failing completely Third-party feed failure: - The Normalizer Agent tracks feed health and flags when a source goes silent - Customer can observe events updates from Events Feed so they will know which feeds are active and their perception coverage is partial. - We use different sources so no single feed failure blinds the system entirely
You're a new startup entering a market with established players like Resilinc and Everstream. What's your technical strategy for winning, not just competing?	We're not trying to out-feature established players on their own terms. Our technical strategy is to own the workflow layer they don't touch: - Resilinc detects. We detect, reason, plan, and execute with a human in the loop - The established players stop at the alert. We start there. Our strategy is to be so deeply embedded in the response workflow that switching means losing institutional memory, not just changing software
How do you think about AI model updates breaking your	Treat model updates: tested, staged, and rollback-ready Every model update should run against our historical incident library before it touches

system? Gemini 2 behaves differently from Gemini 1.5, so how do you manage that?	production. If causal accuracy drops or output schema conformance breaks, we pin to the previous model version until we've adjusted prompts and re-validated.
You're using Gemini today. What's your LLM strategy as the market evolves? Do you go deeper with one provider or stay model-agnostic?	The abstraction layer in our backend means we can swap models, but we're not pretending that's zero cost — prompt engineering is model-specific and our causal reasoning prompts are tuned for Gemini's behavior. → Will try to stay on Gemini since it's the best fit for our context window and ADK integration needs, while maintaining the abstraction so we can move and evaluate annually If a competitor model demonstrably outperforms Gemini on supply chain reasoning benchmarks, then we will start adapting that to our abstraction layer. Long-term plan: fine-tuning a smaller open-source model on our accumulated supply chain incident dataset, which would reduce API costs dramatically and give us a proprietary model nobody else has.
How does your technical roadmap align with how manufacturers are actually transforming digitally?	Mid-market manufacturers are in the middle of a 10-year transformation: ERP modernization, IoT on the factory floor, supplier portal adoption. Most are 3-5 years into that journey, which means they have enough digital infrastructure for us to integrate with but haven't yet consolidated onto a single platform In 5 years, SAP and Oracle will have absorbed most of this functionality into their suites and the window closes. Our technical roadmap is intentionally front-loaded on integration depth: ERP connectors, supplier APIs, logistic platform hooks - because the more deeply we're embedded in their existing digital infrastructure before the platform consolidation happens, the harder we are to displace. We're not betting against SAP long term. We're betting that we can be the best-of-breed layer that SAP integrates with rather than replaces
Could a customer ask you to build a completely custom agent for their specific workflow?	
What does your onboarding process actually look like end to end?	
Does our product allow customization (consulting style)?	

Product roadmap	
Tech propositioning	
Is the notion of integrating into existing ERP feasible? How to make it in practice?	
[product management] estimate productivity improvement by using our product?	

Create full customer profiles

- How do different users use it during demo?
-



	Potential Risk	Likelihood	Mitigations
1	Data Leaks or Cyberattacks	Mid to High	Tenant-isolated vector stores with customer-managed encryption keys; external feed content sandboxed before entering agent pipeline to prevent prompt injectionl role-based access controls
2	Hallucinations	Low to Mid	Human-in-the-loop approval required before any ERP action executes; Hypothesis Agent exposes full causal chain with source citations; regression testing against historycal incident
3	Cloud Outages	Low to Mid	Critical agent outputs cached locally; high-severity events trigger pre-computed emergency from Pattern Memory as fast path; graceful degration to read-only advisory mode during provider downtime
4	Human Misuse	Mid to High	Commit Agent requires cryptographically signed approval token; all agents decisions logged immutably to Audit Agent; role-based permissions separate who can view recommendations from who can approve
5	Degradation at Scale	Low to Mid	Horizontally scalable agent orchestration with isolated per-tenant execution contexts; per-tenant rate limiting prevents noisy-neighbor effects on shared LLM API calls
6	Regulatory & Data Compliance	Low to Mid	Contractual guarantee that customer data never trains shared models; SOC 2 Type II certification roadmap from day one; PIPEDA-compliant data handling with breach notification protocols

Refine # of agents / remove "agent"s from the slides

- Currently: 14 agents
- [ ] Find support data to defend our choice of having many agents
- [ ] Distinguish between **Agentic tasks** (requiring reasoning and decision-making) and **Deterministic tasks** (rule-based processing).

- **Reasoning layer:** merge Cluster Agent and Exposure Agent
    - Into 1 Risk Analyst Agent: takes raw signals → groups → maps them to business exposure
    - Need to refine the prompt + clear context
- **Action layer:**
    - Change Proposal Agent and Audit Agent don't need to be LLMs
        - Turn them into tools → simply call functions like generate erp diff / log to audit trail after it gets human approval
    - Approval Agent should be a tool, UNLESS we want feedback features to refine plans
- **Planning layer:**

cleaned:

Layer	Keep
Perception	Normalization Agent
Reasoning	
Planning	
Action	

- Implementation flow slide: why do we need to go through?
    - In general, show the judges that we didn't hardcode stuff
    - Clearer picture of the data flow in the background
    - But can probably tweak to be more storytelling by putting a persona/customer profile
    - → transition from the demo (how the customer interacts on the surface) to the architecture plans (how the background layers work)
    - The time for this part could be shortened if amy spoke faster (will)
    - Remove repeating the tech stack (python, fast api, supabase) -> more business friendly + won't get technical judges bored.
    - TODO: prepare tech question: how to expand this tech stack as the project scales up? Costs? Security?